



**UNIVERSITÀ DEGLI STUDI
DI GENOVA**

NFT Marketplace: The smart contract report

Course: DECENTRALIZED SYSTEMS

Prepared by :

Musse Gebreezgabher : 6015033

Submitted to :

Professor Marina Ribaudò

Professor Matteo Dell'Amico

February 18, 2025

Smart Contract Report: NFT Marketplace

1. Introduction

This report details the **NFTMarketplace** smart contract, including its **design, functionalities, compilation, and deployment processes**. The contract is written in **Solidity 0.8.20** and follows the **ERC-721 standard** for creating, listing, and trading **Non-Fungible Tokens (NFTs)**. It allows users to mint NFTs, list them for sale, delist them, and execute purchases securely on the blockchain.

2. Smart Contract Overview

The NFTMarketplace contract extends **OpenZeppelin's ERC721URIStorage** for handling NFTs and **Ownable** for ownership management. It leverages **Counters** to generate unique token IDs.

2.1 Contract Imports

The contract imports three essential OpenZeppelin libraries:

```
import "@openzeppelin/contracts/token/ERC721/extensions/ERC721URIStorage.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/utils/Counters.sol";
```

- **ERC721URIStorage**: Provides URI storage for NFTs.
- **Ownable**: Restricts certain functions to the contract owner.
- **Counters**: Ensures unique token IDs for NFTs.

2.2 Data Structures

NFT Struct

The contract maintains a struct to store NFT details:

```
struct NFT {
```

```

uint256 id;
string uri;
address creator;
uint256 price;
bool isListed;
string rarity;
string traits;
}

```

- **id**: Unique identifier for the NFT.
- **uri**: Stores metadata location (IPFS, Arweave, or centralized storage).
- **creator**: Address of the NFT creator.
- **price**: Selling price of the NFT (in Wei).
- **isListed**: Indicates whether the NFT is listed for sale.
- **rarity**: Describes the rarity (e.g., Common, Rare, Legendary).
- **traits**: Stores additional attributes of the NFT.

Mapping for NFT Storage

```
mapping(uint256 => NFT) public nfts;
```

This mapping stores **NFT metadata** using **tokenId** as the key.

2.3 Events

The contract emits the following events to track actions:

```

event Minted(uint256 indexed tokenId, address indexed creator, string uri,
uint256 price, string rarity, string traits);
event Purchased(uint256 indexed tokenId, address indexed buyer, uint256
price);
event Listed(uint256 indexed tokenId, uint256 price);
event Delisted(uint256 indexed tokenId);

```

These events enable **off-chain tracking**, improving transparency and interoperability with external platforms such as **OpenSea** and **Etherscan**.

3. Smart Contract Functionalities

3.1 Minting an NFT

```
function mintNFT(string memory tokenURI, uint256 price, string memory rarity,
string memory traits) public {
    require(price > 0, "Price must be greater than zero");
    _tokenIds.increment();
    uint256 newItemId = _tokenIds.current();
    _mint(msg.sender, newItemId);
    _setTokenURI(newItemId, tokenURI);
    nfts[newItemId] = NFT(newItemId, tokenURI, msg.sender, price, false,
rarity, traits);
    emit Minted(newItemId, msg.sender, tokenURI, price, rarity, traits);
}
```

How It Works:

- Generates a unique `tokenId`.
- Mints the NFT to the sender (`msg.sender`).
- Stores metadata URI, price, rarity, and traits.
- Emits a **Minted** event.

3.2 Listing an NFT for Sale

```
function listNFT(uint256 tokenId, uint256 price) public {
    require(ownerOf(tokenId) == msg.sender, "You must own the NFT to list
it");
    require(price > 0, "Price must be greater than zero");
    nfts[tokenId].price = price;
    nfts[tokenId].isListed = true;
    emit Listed(tokenId, price);
}
```

How It Works:

- Allows NFT owners to set a sale price and list their NFTs.
- Emits a **Listed** event.

3.3 Delisting an NFT

```
function delistNFT(uint256 tokenId) public {
    require(ownerOf(tokenId) == msg.sender, "You must own the NFT to delist
it");
```

```
require(nfts[tokenId].isListed, "NFT is not listed");
nfts[tokenId].isListed = false;
emit Delisted(tokenId);
}
```

How It Works:

- NFT owners can **remove** their NFTs from sale.
- Emits a **Delisted** event.

3.4 Buying an NFT

```
function buyNFT(uint256 tokenId) public payable {
    require(nfts[tokenId].isListed, "NFT is not listed for sale");
    require(msg.value >= nfts[tokenId].price, "Insufficient funds");
    address seller = ownerOf(tokenId);
    _transfer(seller, msg.sender, tokenId);
    nfts[tokenId].isListed = false;
    payable(seller).transfer(msg.value);
    emit Purchased(tokenId, msg.sender, msg.value);
}
```

How It Works:

- Ensures the NFT is listed for sale.
 - Transfers ownership from seller to buyer.
 - Sends payment to the seller.
 - Emits a **Purchased** event.
-

4. Compilation and Deployment Process

4.1 Compiling the Smart Contract

1. Use **Remix IDE** for compilation.
2. **Select Solidity Compiler:** Choose **0.8.20**.
3. Click **"Compile NFTMarketplace.sol"**.

4.2 Deploying the Smart Contract

Using Remix IDE & MetaMask

1. Select "Injected Provider – MetaMask" in Remix.
2. **Deploy Contract:** Click [Deploy](#).
3. **Confirm MetaMask Transaction** (Pays gas fees).
4. **Copy the Deployed Contract Address.**

4.3 Verifying the Contract on Etherscan

1. **Go to Sepolia Etherscan** → Search for the contract address.
 2. **Click "Verify and Publish".**
 3. **Select Compiler Version (0.8.20).**
 4. **Paste the Flattened Contract Code.**
 5. **Submit Verification.**
-

5. Conclusion

The **NFTMarketplace** smart contract enables seamless minting, listing, buying, and delisting of NFTs. It follows the **ERC-721 standard** and integrates with **IPFS and OpenSea** for metadata storage and trading. The contract is deployed on **Ethereum's Sepolia Testnet** and verified on **Etherscan** for transparency and usability.